

Ephemeral Secret Vault

Build a Self-Destructing, Zero-Trace Secret-Sharing Service with Scraper Defense and Atomic Eviction

SECRET INGESTION → ENCRYPTED VAULT → SAFE BURN

STUDENT HANDOUT

Part 0: What This Project Actually Is

0.1 The problem in plain language

Engineers share sensitive credentials every day: temporary database passwords, API keys, private certificates, or production `.env` files. Almost always, these secrets get dropped into corporate chat channels—Slack, Microsoft Teams, Discord, or email.

Once posted, that secret is permanently indexed. It lives forever in searchable chat logs, cloud backups, and notification history. When an account or machine is compromised months later, an attacker searches `password` or `BEGIN RSA PRIVATE KEY` and finds hundreds of active keys.

Right now, this is handled carelessly. People either paste raw secrets in plaintext or use public third-party pastebins that log IP addresses, store secrets unencrypted, and expose data to external scrapers.

Your job: build a standalone, self-destructing secret vault service. It must ingest a plaintext secret via an API or CLI, encrypt it at rest using authenticated symmetric cryptography (AES-256-GCM), assign it a short identifier, and serve it via a time-limited, burn-after-reading link. Once read, or once the expiration timer lapses, the secret is permanently and securely scrubbed from storage, leaving zero trace.

A secret vault is not a generic key-value store. It is a zero-trust, concurrency-hardened service. That is why it must defend against automated link-preview crawlers (which consume burn-on-read links before human recipients open them) and block simultaneous double-read race conditions at the storage engine level.

0.2 What makes this different

Most student web projects end like this: spin up an Express or Flask app, write a `POST /items` and `GET /items/:id`, store plain text in MongoDB or SQLite, close the laptop. That is a basic CRUD tutorial.

This project does not end there. You must deliver:

- **Cryptographic Security at Rest:** Secrets must never touch disk as plaintext. Every entry must have a cryptographically random initialization vector (IV) and an authenticated cipher tag.
- **The Scraper Shield:** Pasting your secret link into Slack, Discord, or WhatsApp must not consume or burn the secret. Automated link-unfurling crawlers must receive only a safe landing screen, reserving the decryption step for an explicit human interaction.
- **Atomic Race-Condition Immunity:** Two parallel requests attempting to read a 1-view secret at the exact same millisecond must not both receive the secret. Exactly one caller must receive the decrypted secret; the second caller must receive a clean `404 Not Found`.
- **Guaranteed Garbage Collection:** Secrets that expire without ever being viewed must be systematically purged by an automated background worker so no residual data persists.

There are two classic ways to fail this:

- **The Naive Burn-on-GET Trap:** A server that burns the secret immediately when `GET /secret/:id` is requested will fail the moment the link is shared in a chat application. A crawler bot will unfurl the link, destroy the secret, and the real recipient will open a dead link.
- **The Double-Read Race Condition:** A server that runs `SELECT * FROM secrets`, decrypts the payload, and then runs `DELETE FROM secrets` will leak the secret twice under high concurrency. You will be tested with parallel requests firing at the same millisecond.

You are evaluated on the entire lifecycle: secret submission, cipher generation, bot defense, atomic destruction, and garbage collection.

Part 1: Important — The Security Model Must Be Real

READ THIS SECTION CAREFULLY. Teams lose marks every time by treating security as an afterthought.

Your vault can be implemented in any modern runtime (Node.js, Go, or Python). What is not acceptable is fake or decorative encryption—such as Base64 encoding, rot13, weak hashing (MD5/SHA256 without encryption), hardcoded global IVs, or storing cryptographic keys in version control.

Why this matters

- **Zero Trust:** If the database file is exfiltrated or dumped, an attacker must find only unreadable ciphertext and random IVs, never plaintext secrets.
- **Cryptographic Authenticity:** If ciphertext is modified or truncated in the database, decryption must throw an authentication failure error rather than emitting corrupted plaintext.
- **Reproducibility:** Judges will fire concurrent `curl` commands, send spoofed crawler User-Agent headers, and inspect database state directly.

If you claim a secret is deleted, it must be gone from storage. A system that simply sets `is_deleted = true` in the database scores zero on the destruction requirement.

Part 2: Before the Session

You will not be given time to debug basic environment setups or package installations during the build. Arrive prepared.

2.1 Set up a scratch repository

- A throwaway local or public repository containing your preferred runtime environment.
- SQLite tools installed locally (`sqlite3` CLI) to inspect table states during debugging.
- Docker installed if you choose an external cache or database, though a local embedded database (SQLite with WAL mode) is strongly recommended for speed.

2.2 Pre-pull what you will need

Ensure your language's standard crypto libraries and SQLite drivers are verified:

- **Node.js (v20+):** `better-sqlite3`, native `node:crypto`.
- **Python (3.11+):** `cryptography`, native `sqlite3`.
- **Go (1.21+):** `crypto/cipher`, `crypto/aes`, `modernc.org/sqlite`.

Verify that your HTTP test runner (such as `curl`, `autocannon`, or a custom Python test script) is ready to send parallel requests.

2.3 Bring test scenarios

Prepare three test scenarios at home so you are not scrambling:

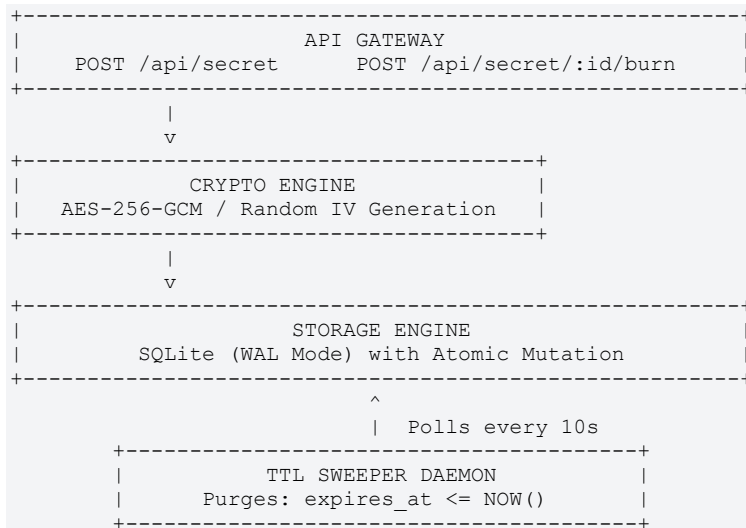
- **Scenario A (Happy Path):** A secret created with a 5-minute TTL and 1 view. Verified, decrypted once, and confirmed completely absent on a second query.
- **Scenario B (The Link Crawler):** A secret fetched with a `User-Agent: Slackbot-LinkExpanding 1.0` header. The bot receives an HTML shell; the secret remains unburned.
- **Scenario C (Concurrent Attack):** 20 simultaneous HTTP calls requesting the same 1-view secret at the same millisecond. Exactly 1 receives the decrypted secret; 19 receive an HTTP 404.

2.4 A stack that fits the clock

Layer	Suggested	Note
Runtime	Node.js (Express/Fastify) or Go	Fast startup, built-in crypto primitives, minimal boilerplate.
Storage	SQLite in WAL mode (<code>PRAGMA journal_mode = WAL;</code>)	Zero network overhead, ACID-compliant, native support for atomic transactions.
Encryption	AES-256-GCM	Authenticated encryption: guarantees both confidentiality and data integrity.
Identifier	nanoid or <code>crypto.randomBytes(8).toString('hex')</code>	Generates short, URL-friendly, non-guessable access slugs.
Sweeper	In-process background timer (<code>setInterval</code> or Go ticker)	Runs an atomic cleanup query every 10–30 seconds.

Part 3: What You Are Building

Three internal components operating on one unified storage model:



1. **Crypto Engine:** Encrypts incoming plaintext with a 256-bit key and generates a unique, cryptographically secure 96-bit (12-byte) initialization vector (IV) for every secret. Produces ciphertext along with a 128-bit authentication tag.
2. **Storage Engine:** Holds encrypted records and exposes atomic read-and-decrement operations to guarantee that `views_remaining` never drops below zero and cannot be double-read.
3. **Sweeper Daemon:** Periodically purges unread records whose `expires_at` timestamp has passed, freeing resources and ensuring expired data does not linger on disk.

Part 4: The Interface Contract

Your service must strictly implement the following API contracts.

1. Ingestion Endpoint

POST /api/secret

Request:

```
{
  "secret": "my-database-password-xyz",
  "ttl_seconds": 3600,
  "max_views": 1
}
```

Response (HTTP 201 Created):

```
{
  "id": "e8a1b4c7d2e9",
  "view_url": "http://localhost:3000/view/e8a1b4c7d2e9",
  "expires_at": "2026-09-24T12:00:00.000Z",
  "views_remaining": 1
}
```

2. Human Landing Endpoint (Safe Splash Screen)

GET /view/:id

Response (HTTP 200 OK): Returns an HTML document. Does **not** decrypt the secret. Displays metadata:

- Notice: *"You have been sent a secure, self-destructing secret."*
- Expiration timestamp and views remaining.
- A prominent interactive button: **"Reveal and Destroy Secret"** which executes a client-side POST to /api/secret/:id/burn.

3. Destruction & Retrieval Endpoint

POST /api/secret/:id/burn

Response (HTTP 200 OK — Success):

```
{
  "secret": "my-database-password-xyz",
  "views_remaining": 0,
  "burned": true
}
```

Response (HTTP 404 Not Found — Burned, Expired, or Non-Existent):

```
{
  "error": "Secret not found, expired, or already destroyed."
}
```

Part 5: The Concurrency & Scraper Rules (Mandatory)

5.1 The Scraper Defense Rule

When a link is posted to Slack, Discord, or Teams, their backend servers trigger a GET request to display an OpenGraph preview card.

- If your application burns the secret on a plain `GET` request, the automated preview bot destroys it before the user ever sees it.
- **The Rule:** The `GET /view/:id` endpoint must **never** decrement views or delete the secret. It must serve only static HTML and metadata.
- Decryption and destruction must be exclusively executed via `POST /api/secret/:id/burn`.
- **Crawler User-Agent Gate:** Any `GET` or `POST` request carrying known bot signatures (e.g., `*bot*`, `*crawl*`, `*spider*`, `*Slackbot*`, `*facebookexternalhit*`) must be rejected or served a blank response without modifying state.

5.2 The No-Double-Read Rule (Zero Partial Credit)

A secret configured with `max_views: 1` must **never** be delivered twice under any condition.

If 20 threads request the burn endpoint for the same secret simultaneously:

- Exactly **one** request must receive the decrypted secret and `HTTP 200`.
- The remaining **19** requests must receive `HTTP 404`.

A system where two simultaneous callers both receive the decrypted secret fails the evaluation, receiving **0 marks** for concurrency.

Part 6: How to Actually Build This

6.1 Keep the schema minimal

Do not over-engineer users, accounts, or auth roles unless you finish early. The core database requires only one table:

```
CREATE TABLE IF NOT EXISTS secrets (
  id TEXT PRIMARY KEY,
  ciphertext BLOB NOT NULL,
  iv BLOB NOT NULL,
  auth_tag BLOB NOT NULL,
  max_views INTEGER NOT NULL DEFAULT 1,
  views_remaining INTEGER NOT NULL DEFAULT 1,
  expires_at INTEGER NOT NULL,
  created_at INTEGER NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_secrets_expiry ON secrets(expires_at);
```

6.2 Build the pipe before the logic

Start with simple, verifiable steps:

1. Build the HTTP server and verify that `POST /api/secret` writes a dummy unencrypted string to SQLite and returns an ID.
2. Verify that `POST /api/secret/:id/burn` deletes the row from SQLite.
3. Once this plumbing works, integrate real AES-256-GCM encryption and decryption.

6.3 Use atomic SQL updates

Do not separate the view check and the view decrement into distinct SQL calls.

Vulnerable Implementation:

```
// BROKEN: Vulnerable to race conditions
const row = db.prepare("SELECT * FROM secrets WHERE id = ?").get(id);
if (row && row.views_remaining > 0) {
  db.prepare("UPDATE secrets SET views_remaining = views_remaining - 1 WHERE id = ?").run(id);
  return decrypt(row.ciphertext);
}
```

Secure, Atomic Implementation:

```
// SECURE: Atomic check and decrement in a single statement
const row = db.prepare(`
  UPDATE secrets
  SET views_remaining = views_remaining - 1
  WHERE id = ?
  AND views_remaining > 0
  AND expires_at > ?
  RETURNING ciphertext, iv, auth_tag, views_remaining;
`).get(id, Date.now());

if (!row) {
  return res.status(404).json({ error: "Secret unavailable or expired." });
}

if (row.views_remaining === 0) {
  // Immediate hard cleanup
  db.prepare("DELETE FROM secrets WHERE id = ?").run(id);
}

const plaintext = decrypt(row.ciphertext, row.iv, row.auth_tag);
return res.status(200).json({ secret: plaintext });
```

Part 7: Requirements

7.1 Must-Have (Pass Criteria)

No.	Requirement
1	Secrets are encrypted at rest using AES-256-GCM; database exfiltration yields zero plaintext.
2	Every secret generated uses a unique, cryptographically random initialization vector (IV).
3	Secrets are assigned a short, non-enumerable, URL-safe unique identifier.
4	GET /view/:id serves an HTML splash screen and does not burn the secret.
5	Secrets burn only on an explicit POST /api/secret/:id/burn call.
6	A secret set to 1 view cannot be read twice, even under concurrent requests.
7	Expired secrets cannot be read and are automatically removed by a background worker.
8	Decrypting tampered or truncated ciphertext fails cleanly without leaking stack traces.
9	Server returns appropriate HTTP status codes (201 Created, 200 OK, 404 Not Found).
10	A minimal CLI script or curl command is provided to push a secret directly from terminal stdin.

7.2 Stretch (Distinction Criteria)

- **Client-Side End-to-End Encryption:** Secret is encrypted in the browser/CLI via WebCrypto before transmission; the server stores ciphertext and never possesses the decryption key (passphrase passed as URL hash fragment `#key`).
- **Password Protection:** Optional passphrase required to unlock the secret upon burn.
- **Audit Fingerprint:** Returning a cryptographic checksum (SHA-256) of the secret to the sender at creation time for integrity verification.
- **High-Load Performance:** Under `autocannon` or `k6` benchmark, service sustains >1,500 reads/sec with p99 < 15 ms.

Part 8: Timeline (3 Hours)

Time	Phase	You are done when...
0:00 – 0:15	Architecture & Repo Setup	SQLite initialized with schema, health-check endpoint responds 200 OK.
0:15 – 0:45	Crypto Core	Tested functions: <code>encrypt(plaintext) -> {ct, iv, tag}</code> and <code>decrypt(ct, iv, tag) -> plaintext</code> .
0:45 – 1:25	Core Endpoints	<code>POST /api/secret</code> stores encrypted rows; <code>POST /api/secret/:id/burn</code> decrypts and wipes records.
1:25 – 1:55	Scraper Defense & Landing Page	<code>GET /view/:id</code> serves HTML; interactive button triggers burn; bot User-Agents are handled.
1:55 – 2:25	Concurrency Hardening & Sweeper	Atomic update query passes 20 parallel requests test; background sweeper purges expired records.
2:25 – 2:40	Hostile Edge-Case Testing	Run bad inputs, corrupt tokens, and scraper simulations. Verify checklist.
2:40 – 2:45	BUILD FREEZE	Code is committed and pushed. Nothing further is touched.
2:45 – 3:00	Report	<code>REPORT.md</code> is finalized and committed.

THE 2:45 FREEZE IS HARD. Work that is not pushed does not exist.

Part 9: The Report (15 Minutes)

One file: `REPORT.md` in your repository.

1. **Architecture Overview:** Concise summary of the data flow, database schema, and crypto primitives chosen.
2. **Security & Threat Model:** How AES-256-GCM is implemented, how keys/IVs are managed, and why Base64 or simple hashing was rejected.
3. **The Scraper Problem:** How the system guarantees link-preview crawlers do not prematurely consume secrets.
4. **Concurrency Strategy:** How simultaneous read requests are handled at the database level.
5. **Results & Benchmarks:** A markdown table summarizing tests against:
 - Happy path (1-view burn).

- Simultaneous race condition (20 parallel requests).
 - TTL expiration cleanup.
 - Invalid or tampered cipher payload.
6. **Limitations & Next Steps:** Concrete improvements (e.g., zero-knowledge client-side encryption, rate limiting).
 7. **Quickstart Instructions:** Exact commands to install dependencies, run tests, and execute a secret transaction via `curl`.

Part 10: Pre-Freeze Checklist

At 2:25, stop adding features and verify every line:

- ☐ Database contains zero plaintext secrets after ingestion.
- ☐ Every encrypted record has a distinct IV.
- ☐ `GET /view/:id` does not decrement views or delete the secret.
- ☐ Automated crawlers (Slackbot, Twitterbot) cannot consume the secret.
- ☐ 20 simultaneous requests against a 1-view secret result in exactly one `200 OK` and nineteen `404 Not Found`.
- ☐ Background sweeper deletes expired records automatically.
- ☐ Corrupted ciphertext returns a clean `404` or `400` without leaking stack traces.
- ☐ Terminal CLI pipeline (`cat secret.txt | ./vault-cli`) works as documented.
- ☐ `REPORT.md` is committed.
- ☐ All code pushed to the target repository.

Part 11: Marking Criteria

Component	Marks
Cryptographic Rigor: AES-256-GCM, unique IVs per record, zero plaintext on disk	20
Concurrency Defense: Atomic decrement/delete; zero double-reads under parallel load	20
Scraper Shield: Separation of viewing vs burning; crawler resistance	15
Garbage Collection: Automated, reliable cleanup of expired records	15
API Quality & Ergonomics: Clean JSON schemas, proper status codes, working CLI helper	10
Hostile Input Resilience: Clean handling of tampered data, bad IDs, and missing keys	10
Report: Architecture, Honesty & Test Results: Clear <code>REPORT.md</code> documentation	10
Total	100

Part 12: Frequently Asked Questions

Do we need an external database like PostgreSQL or Redis?

No. In fact, using SQLite in WAL mode is preferred: it requires zero external setup, runs entirely in-memory or in a single local file, and provides strict ACID transactions out of the box.

Can we implement client-side encryption?

Yes, but treat it as a stretch goal. Build server-side AES-256-GCM first. Once your pipeline passes the concurrency and scraper tests, you can add client-side WebCrypto encryption.

What happens if the client closes the browser on `/view/:id` without clicking Reveal?

Nothing. The secret remains encrypted and available in storage until its TTL expires or someone clicks the Reveal button.

Can we store the AES master key in the database table?

Never. The master encryption key must be loaded from an environment variable (`VAULT_MASTER_KEY`) or generated at application boot. Storing the key next to the ciphertext defeats the purpose of encryption at rest.